Réception du matériel

Nous avons récupéré le matériel commandé. Seule la carte ESP32-S3 est pour l'instant disponible ; les autres composants (IMU, INA226, BME280, MS5837, ESC, moteurs, servos, seringue de ballast) seront livrés plus tard. J'ai donc choisi d'écrire dès maintenant l'intégralité du firmware, en isolant les fonctions de lecture physique derrière des fonctions d'abstraction. Ainsi, l'arrivée des capteurs ne demandera qu'un remplacement local, sans refonte du reste du code.

Création du repos Github :

J'ai créé un dépôt GitHub unique. Le dossier firmware/ accueillera plus tard un second sous-dossier pour le code de la Raspberry Pi 5. J'ai également ajouté un fichier .gitignore excluant les fichiers de compilation, les configurations locales de l'IDE et les éventuels secrets.

Programmation :

Plutôt que d'écrire tout le code dans un unique fichier .ino, j'ai opté pour une programmation modulaire. Chaque fonctionnalité est isolée dans un module composé de deux fichiers :

- un fichier .h qui déclare l'interface publique (types et prototypes) ;
- un fichier .cpp qui contient l'implémentation réelle, invisible pour les autres modules.

Cette approche permet de compiler chaque module indépendamment, de faire évoluer une implémentation sans toucher aux fichiers appelants, et d'éviter les conflits Git lorsque deux personnes travaillent en parallèle.

config.h centralise toutes les constantes du projet : broches, adresses I2C, fréquences des tâches, gains PID et seuils de sécurité. Chaque valeur est commentée avec son unité et sa justification. Ce fichier est le point d'entrée unique pour tout réglage : aucune valeur n'apparaît en dur ailleurs.

Le module sensors (sensors.h + sensors.cpp) regroupe l'acquisition : IMU 9 axes, trois INA226 (un par pack de batteries), BME280 pour l'humidité interne et MS5837 pour la profondeur. Il expose trois fonctions publiques (sensors_lire_imu, sensors_lire_alim, sensors_lire_env) et intègre un filtre de Kalman 1D pour le roll et le pitch, qui fusionne accéléromètre et gyroscope. Le cap est estimé par intégration du gyroscope avec correction progressive par le magnétomètre. La lecture physique de l'IMU est isolée dans une fonction interne imu_lire_brut qui, tant que le composant n'est pas reçu, retourne false.